

课程笔记

CS231N 2025 第 3 讲：正则化与优化

Codex Harness Worker 1

2026-04-15



视频作者/频道：Stanford Online

发布日期：2025-09-02

视频时长：1:08:39

视频链接：<https://www.youtube.com/watch?v=dyNGd06MWn4>

目录

1	目标函数：经验风险最小化加上复杂度控制	2
1.1	本章小结	2
2	为什么要正则化：泛化而非死记硬背	2
2.1	本章小结	3
3	如何得到梯度：数值法与解析法	3
3.1	本章小结	3
4	从全量梯度下降到小批量 SGD	3
4.1	本章小结	4
5	从 SGD 到 Momentum，再到 Adam	4
5.1	本章小结	5
6	学习率日程与二阶方法的角色	5
6.1	本章小结	6
7	总结与延伸	6
7.1	拓展阅读	6

1 目标函数：经验风险最小化加上复杂度控制

上一讲已经把图像分类写成“分数函数 + 损失函数”的形式，这一讲则进一步把训练目标写成完整优化问题。课程最强调的统一表达是：

$$L(W) = \frac{1}{N} \sum_{i=1}^N L_i(f(x_i; W), y_i) + \lambda R(W).$$

其中， L_i 是第 i 个样本的数据损失， $R(W)$ 是正则项 (regularizer)， λ 控制正则强度。这个式子看似简单，却是后续几乎所有深度学习训练问题的原型。

符号上， N 表示训练样本数， $f(x_i; W)$ 是由参数 W 给出的预测分数， y_i 是真实标签， $R(W)$ 则衡量模型复杂度。教材化理解时，务必把它拆成两个问题：第一，模型是否足够贴合训练数据；第二，模型是否因为过度复杂而失去泛化能力。正则化正是在第二个问题上发挥作用。

Regularization

λ = regularization strength
(hyperparameter)

$$L(W) = \underbrace{\frac{1}{N} \sum_{i=1}^N L_i(f(x_i, W), y_i)}_{\text{Data loss: Model predictions should match training data}} + \underbrace{\lambda R(W)}_{\text{Regularization: Prevent the model from doing too well on training data}}$$

Data loss: Model predictions should match training data

Regularization: Prevent the model from doing too well on training data

Why regularize?

- Express preferences over weights
- Make the model simple so it works on test data
- Improve optimization by adding curvature

Stanford CS231n 10th Anniversary

Lecture 3 - 23

April 8, 2025

图 1: 损失由数据项与正则项组成，正则化强度控制“拟合训练集”与“保持参数简单”之间的平衡。

正则化与数据损失共同定义了真正要最小化的目标函数。深度学习训练从来不是单目标，而是拟合与泛化之间的权衡。

1.1 本章小结

2 为什么要正则化：泛化而非死记硬背

正则化 (regularization) 最常见的直觉，是“防止模型把训练集背下来”。但更精确地说，正则化是在向模型注入偏好：偏好更小的权重、更平滑的函数、更少的自由度，或者更稳定的预测行为。对于线性分类器，这常常体现为 L_2 范数惩罚；对于神经网络，则还会体现在 dropout、数据增强、早停 (early stopping) 等更丰富的策略上。

课程在这里特别强调一个容易被忽略的事实：较低的训练误差并不保证较低的测试误差。真正的目标是测试分布上的表现，而不是训练分布上的记忆能力。因此，判断一个优化策略是否有效，必须看它在验证集上的行为，而不是只盯着训练 loss 曲线。

正则化不是训练失败时才加的补丁，而是现代模型设计的组成部分。泛化能力必须通过验证集与测试集来检验，而不能从训练误差直接推断。

2.1 本章小结

3 如何得到梯度：数值法与解析法

一旦目标函数写出来，下一步就是求梯度。课程先用有限差分（finite differences）解释数值梯度：

$$\frac{\partial L}{\partial w_k} \approx \frac{L(w_k + h) - L(w_k - h)}{2h}.$$

其中， h 是一个很小的扰动， w_k 是某个参数。这个公式的意义在于：即便你不会手推导数，也能通过“微小扰动前后的 loss 变化”近似得到梯度。

但数值梯度只适合做 gradient check。它的优点是通用、容易实现；缺点是极慢，而且会受数值误差影响。真正训练模型时，我们必须依赖解析梯度（analytic gradient），也就是用微积分直接推导每个参数对 loss 的偏导数。后面反向传播的全部价值，就体现在“如何高效得到解析梯度”。

current W:	W + h (third dim):	gradient dW:
[0.34, -1.11, 0.78, 0.12, 0.55, 2.81, -3.1, -1.5, 0.33,...]	[0.34, -1.11, 0.78 + 0.0001, 0.12, 0.55, 2.81, -3.1, -1.5, 0.33,...]	[-2.5, 0.6, 0, ?, ?,...]
loss 1.25347	loss 1.25347	Numeric Gradient - Slow! Need to loop over all dimensions - Approximate

图 2: 数值梯度示意：通过有限差分近似偏导数。它常用于 gradient check，但训练时过慢。

有限差分是检查梯度是否写对的工具，不是训练大模型的方法。训练时必须依赖可组合的解析梯度，这直接引向反向传播。

3.1 本章小结

4 从全量梯度下降到小批量 SGD

理论上，最直接的更新规则是全量梯度下降：

$$W \leftarrow W - \eta \nabla_W L(W),$$

其中, η 是学习率 (learning rate), $\nabla_W L(W)$ 是对全体训练集求出的梯度。但在大规模数据下, 全量梯度每一步都要扫完整个数据集, 代价太高, 因此实际训练通常使用小批量随机梯度下降 (mini-batch SGD)。

mini-batch SGD 用一个小批样本近似整体梯度。这样得到的更新方向带有噪声, 但这种噪声并不总是坏事: 它能降低每步成本, 也常常帮助模型逃离某些坏的局部结构。课程在这里想传达的关键信息是: 优化不是只看“每一步有多准”, 还要看“在固定算力预算下能走多少步”。

SGD 把每步更新从“极准但极贵”改成“略噪但高效”, 这是深度学习可扩展性的关键。学习率决定每步走多远, 而批大小决定梯度估计有多噪。

4.1 本章小结

5 从 SGD 到 Momentum, 再到 Adam

纯 SGD 的一个问题是, 在狭长谷地中常常左右震荡、前进缓慢。动量法 (momentum) 通过累积历史梯度来缓解这个问题:

$$v_t = \mu v_{t-1} - \eta \nabla_W L(W_t), \quad W_{t+1} = W_t + v_t.$$

其中, v_t 是速度项, μ 是动量系数。它的直觉很像在优化地形上滚动一个有惯性的球: 沿一致方向会不断加速, 来回震荡方向则会相互抵消。

SGD + Momentum:

continue moving in the general direction as the previous iterations

SGD

$$x_{t+1} = x_t - \alpha \nabla f(x_t)$$

```
while True:
    dx = compute_gradient(x)
    x -= learning_rate * dx
```

SGD+Momentum

$$v_{t+1} = \rho v_t + \nabla f(x_t)$$

$$x_{t+1} = x_t - \alpha v_{t+1}$$

```
vx = 0
while True:
    dx = compute_gradient(x)
    vx = rho * vx + dx
    x -= learning_rate * vx
```

- Build up “velocity” as a running mean of gradients
- Rho gives “momentum”; typically rho=0.9 or 0.99

Sutskever et al, “On the importance of initialization and momentum in deep learning”, ICML 2013

Stanford CS231n 10th Anniversary

Lecture 3 - 64

April 8, 2025

图 3: 动量法把历史梯度累积成“速度”, 从而在狭长谷地中减少震荡、加速沿主方向前进。

自适应方法则进一步试图为不同参数分配不同步长。以 Adam 为例, 可写成

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2,$$

$$W_{t+1} = W_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}.$$

其中, g_t 是当前梯度, m_t 与 v_t 分别是一阶矩和二阶矩的指数滑动平均, $\hat{m}_t, \hat{v}_t$ 是偏差校正后的估计, ϵ 用于数值稳定。这个公式说明 Adam 通过参数级步长调整, 适应不同尺度和稀疏度的更新。

Adam (full form)

```
first_moment = 0
second_moment = 0
for t in range(1, num_observations):
    dx = compute_gradient(x)
```

```
    first_moment = beta1 * first_moment + (1 - beta1) * dx
```

```
    second_moment = beta2 * second_moment + (1 - beta2) * dx * dx
```

```
    first_unbias = first_moment / (1 - beta1 ** t)
```

```
    second_unbias = second_moment / (1 - beta2 ** t)
```

```
    x -= learning_rate * first_unbias / (np.sqrt(second_unbias) + 1e-7))
```

Momentum

Bias correction

AdaGrad / RMSProp

Bias correction for the fact that first and second moment estimates start at zero

Adam with **beta1 = 0.9**, **beta2 = 0.999**, and **learning_rate = 1e-3 or 5e-4** is a great starting point for many models!

Kingma and Ba, "Adam: A method for stochastic optimization", ICLR 2015

Stanford CS231n 10th Anniversary

Lecture 3 - 74

April 8, 2025

图 4: Adam 结合一阶矩与二阶矩的指数滑动平均, 在不同参数尺度上自适应调整步长。

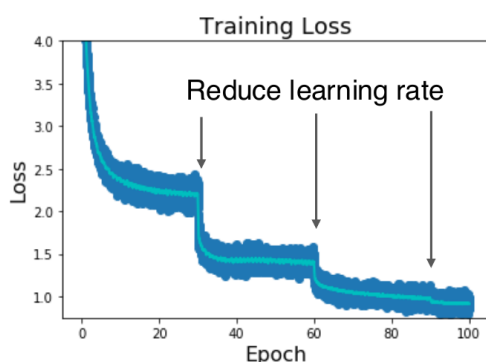
Momentum 通过历史速度减少震荡、提升主方向推进效率。Adam 则通过一阶矩和二阶矩估计, 实现参数级自适应步长。

5.1 本章小结

6 学习率日程与二阶方法的角色

即使优化器固定, 学习率日程 (learning rate schedule) 仍然决定了训练能否真正收敛。课程展示了 step decay 等思路, 其共同逻辑是: 训练早期需要较大学习率快速探索, 训练后期则需要更小步长把参数稳定压到更优区域。

Learning rate decays over time



Step: Reduce learning rate at a few fixed points. E.g. for ResNets, multiply LR by 0.1 after epochs 30, 60, and 90.

Stanford CS231n 10th Anniversary

Lecture 3 - 84

April 8, 2025

图 5: 学习率衰减策略决定了训练后期能否稳定收敛; 它通常与动量和批大小共同作用。

在理论上，二阶方法利用 Hessian 信息能够更精确描述局部曲率，但在深度网络中往往因存储和计算开销过高而难以直接使用。因此实践中的主流方案仍是“一阶优化器 + 合理学习率调度 + 正则化”。

学习率是训练动力学中最敏感的超参数之一，日程设计常常和优化器本身同等重要。深度学习实践仍以一阶方法为主，二阶方法更多作为理解曲率和局部几何的参考。

6.1 本章小结

7 总结与延伸

Lecture 3 解决的是一个朴素但决定性的问题：写下目标函数之后，怎样把参数真正学出来，而且学出来的模型还能泛化。为此，课程把训练拆成四个层次：目标定义、正则化、梯度求法、更新策略。只有四者协同，训练过程才是完整的。

从教材角度看，这一讲的地位极高，因为后续神经网络、卷积网络、自监督学习乃至大模型训练，都没有跳出这个框架。目标函数可能换，模型结构可能变，但“正则化 + 梯度 + 优化器 + 学习率日程”的组合始终存在。下一讲进入神经网络与反向传播时，读者应该把本讲内容当作它的优化背景板。

7.1 拓展阅读

- Optimization